

如何通过中间件定制 Agent 运行框架

How Middleware Lets You Customize Your Agent Harness

LangChain · Sydney Runkle · 2026-03-26 · 原文 / Source

完整译文与英文逐段对照。依据你提供的网页存档翻译，保留原文的观点、例子与限定条件。图表（含图注）、代码及其中的文字维持原文；网页导航、订阅表单和站点页脚不属于文章正文。中文为学习用非官方译文。

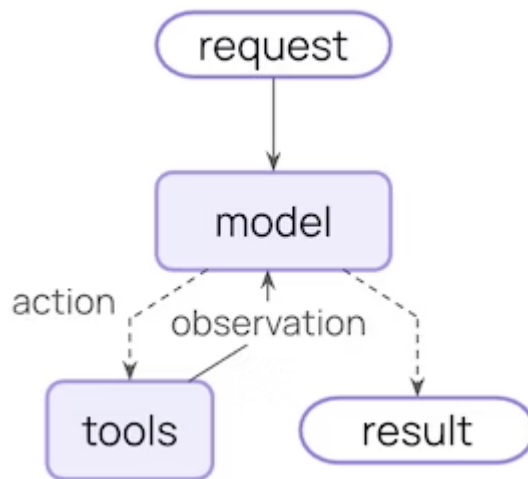
Agent 运行框架帮助我们构建 Agent：它将 LLM 与环境连接起来，让模型能够采取行动。

构建 Agent 时，你很可能希望拥有一个针对具体应用的运行框架。“Agent 中间件”让你能够以 LangChain 和 Deep Agents 的坚实基础为起点，再根据自己的使用场景进行定制。

什么是 Agent 运行框架

Agent 是围绕模型构建的系统。模型需要连接环境、数据、记忆与工具。Agent 运行框架，就是帮助你完成这些连接的系统。

每个 Agent 运行框架的核心都相同，而且非常简单：一个在循环中运行、不断调用工具的 LLM。虽然简单，这个核心循环却蕴含着强大的能力。



LangChain 提供了 `create_agent`，它是仅包含这一核心循环的抽象。

为什么要定制 Agent 运行框架

不同的 Agent 使用场景有不同需求，也可能需要不同的运行框架。

运行框架的某些部分，例如指令或工具，很容易定制。比如，LangChain 中的 `create_agent` 允许你传入系统提示词和工具。

其他部分就更复杂一些。如果你希望每次模型执行前，都先运行一个指定步骤，该怎么办？如果你希望每次都检查工具输出中是否存在某些内容，又该怎么办？

涉及修改 Agent 核心循环的内容，更难调整。但如果处理得当，就能在继续利用核心运行框架的同时，实现非常强大的定制。

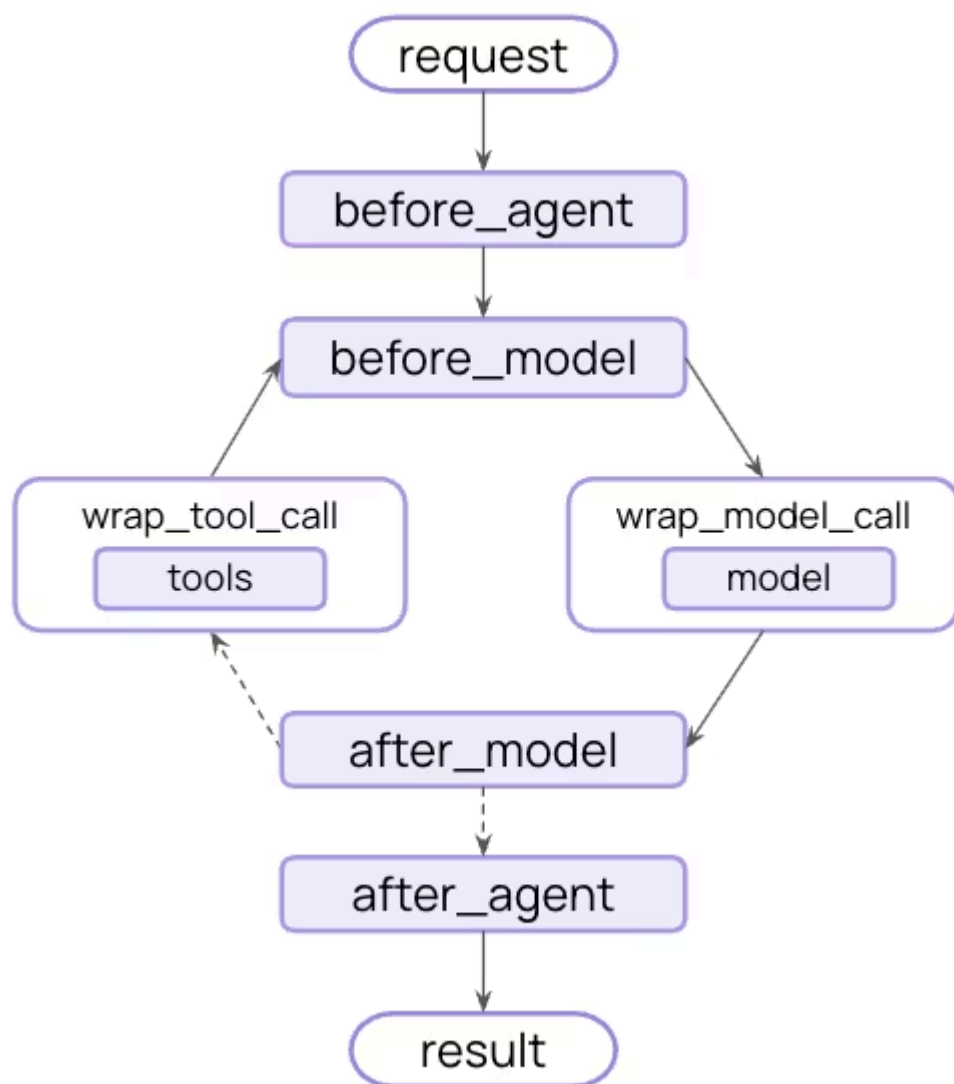
`AgentMiddleware` 就是我们对此的答案，也是我们让开发者定制 LangChain Agent 的方式。

什么是 Agent 中间件？



“中间件”是其他软件工程实践中也经常使用的通用术语。不过，下文讨论的是一种不同的系统，我们称它为 Agent 中间件。

中间件提供一组钩子，让你在每个步骤执行前后运行自定义逻辑，从而控制循环中每个阶段发生什么：



- **before_agent**：在 Agent 被调用时运行一次。适合加载记忆、连接资源，或验证初始输入。
- **before_model**：每次模型调用前触发。可以用来裁剪历史记录，或在个人身份信息（PII）传入 LLM 前将其检测出来。

- `wrap_model_call`：从头到尾包装模型调用。缓存、重试，以及动态调整模型请求（例如改变可用工具），都在这里处理。
- `wrap_tool_call`：以类似方式包装工具执行。可以注入上下文、拦截结果，或控制哪些工具实际获准执行。
- `after_model`：在模型响应后、工具执行前运行。这是引入人类介入最自然的位置。
- `after_agent`：在完成时运行一次。可以保存结果、发送通知、清理资源。

中间件可以组合，因此你可以按需灵活搭配。

LangChain 内置了一组用于常见模式的中间件，例如摘要、重试与 PII 隐去。开发者还可以继承 `AgentMiddleware` 类，为业务特有的需求编写自己的中间件。

中间件示例

定制需求往往围绕相似的主题展开。下面是最常见的使用场景：

业务逻辑与合规。有些事情不能只靠提示词处理，例如 PII 隐去和内容审核。这些是必须每次都执行的确定性策略。你不能只靠提示词就满足 HIPAA 合规要求。

深入了解：PII 检测

LangChain 内置的 `PIIMiddleware` 实现了 `before_model` 和 `after_model` 钩子。它能够对模型输入、模型输出和工具输出中的 PII 进行遮蔽、隐去或哈希处理。对于最关键的 PII 检测情形，它还可以抛出 `PIIDetectionError`。

动态控制 Agent。中间件可以在运行时调整 Agent：根据当前状态注入工具、在任务进行中更换模型，或者随着上下文变化更新系统提示词。这意味着主动控制 Agent 在每一步的行为。

深入了解：动态工具选择

LangChain 的 `LLMToolSelectorMiddleware` 在 `wrap_model_call` 钩子中运行一个速度较快的 LLM，判断工具注册表中哪些工具与当前请求相关。随后，它将这些工具绑定到模型请求上，尽量减少主模型调用中不必要工具造成的上下文膨胀。

上下文管理。模型的表现取决于你提供给它的内容。例如，接近 token 上限时，你可能需要生成摘要，并裁剪充满噪声的工具输入与输出。上下文工程是运行时的问题，而不是一次性编写提示词的问题。

深入了解：摘要与上下文卸载

LangChain 内置的 `SummarizationMiddleware` 实现了 `before_model` 钩子。为避免上下文溢出，当消息历史超过一定 token 阈值时，它会先对内容进行摘要，再传给模型。这个中间件的扩展版本会实现 `wrap_tool_call` 钩子，将冗长的工具调用输入与输出转存到文件系统。

为生产环境做好准备。中间件让你能够内置模型 / 工具重试逻辑、模型回退，以及通过中断实现的人类介入。这些功能在演示中不一定显眼，却是生产环境中的 Agent 必不可少的。

深入了解：模型重试

LangChain 内置的 `ModelRetryMiddleware` 实现了 `wrap_model_call` 钩子，用重试处理器包装模型的 API 调用。这个处理器支持重试次数、退避系数，以及初始延迟等重试配置，以应对速率限制问题。

工具集。注入那些需要在 Agent 循环前后进行自定义初始化和清理的工具，例如连接外部工具服务器、初始化 Shell，或启动沙箱。

深入了解：Shell 工具中间件

LangChain 的 `ShellToolMiddleware` 实现了 `before_agent` 和 `after_agent` 钩子，在核心 Agent 循环前后初始化并释放 Shell 资源。它还会把 Shell 工具加入模型的工具列表。

Deep Agents 案例研究

Deep Agents 是一个功能齐备、开箱即用的 Agent 运行框架。它完全构建在 LangChain 用于创建 Agent 的标准入口 `create_agent` 之上，并在上层加入了一组体现既定设计取舍的中间件。

下面是支撑 Deep Agents 的几个中间件：

- `FilesystemMiddleware`：基于文件的上下文加载、卸载与长期记忆
- `SubagentMiddleware`：具有上下文隔离的子 Agent
- `SummarizationMiddleware`：长时间任务的上下文溢出管理
- `SkillsMiddleware`：专门能力的渐进式披露
- 以及更多功能！

如需全面了解支撑 Deep Agents 的中间件，请参阅这份指南，以及 Vivek 撰写的《剖析 Agent 运行框架》。

在上述能力的基础上，你还可以向 Deep Agents 添加更多中间件，让它适应自己的使用场景！

为什么我们看好 Agent 中间件

模型正在变得更强，这会改变中间件栈中的一部分。Deep Agents 如今承担的一些工作——摘要、工具选择、输出裁剪——最终会被模型本身吸收。

但底层需求不会改变。开发者始终需要进行定制的手段：确定性地执行策略、为生产使用提供防护机制，以及实现特定场景的业务逻辑。这些都不会转移到模型内部。它们仍然属于运行框架，而中间件依旧是将这些能力开放出来的最清晰方式。

自 LangChain v1 发布以来，我们已经看到了这一点。中间件让不同团队负责不同事项，让业务逻辑与 Agent 核心代码解耦，并且便于在整个组织内复用逻辑。完全基于中间件构建 Deep Agents 的经历，让我们确信这是一种正确的抽象。

想从最精简的 Agent 运行框架开始？试试 `create_agent` 中的中间件。

想在更完善的 Agent 运行框架上构建？试试 `create_deep_agent` 中的中间件。

想贡献自己编写的中间件？请查看这里的指南。